

Assignment 2: Data Structures & In-Memory Workload Engine

Student: Temirlan Zhangirkhan

Group: SE 2521

Github:  GitHub - Legwnsa/DAA_Assignment2

1) Asymptotic Complexity Analysis

Below is the theoretical analysis for all implemented operations in DynamicArray, MyLinkedList, and MinHeap

Structure	Operation	Best Case	Average Case	Worst Case	Auxiliary Space	Justification
DynamicArray	add(x)	$\Omega(1)$	$\Theta(1)$	$O(N)$	$O(1)$	Amortized $O(1)$; worst case $O(N)$ occurs when resizing (2x capacity expansion) is triggered.
	add(index, x)	$\Omega(1)$	$\Theta(N)$	$O(N)$	$O(1)$	Inserting at tail is $O(1)$; head/middle insertions require right-shifting existing elements.
	remove(index)	$\Omega(1)$	$\Theta(N)$	$O(N)$	$O(1)$	Removing tail element is $O(1)$; head/middle removals require left-shifting remaining elements.
	get(index)	$\Omega(1)$	$\Theta(1)$	$O(1)$	$O(1)$	Direct random access calculated via base memory address offset: $\text{base} + \text{index} * \text{size}$.
	contains(x)	$\Omega(1)$	$\Theta(N)$	$O(N)$	$O(1)$	Sequential scan over contiguous array elements; best case when

						target is at
MyLinkedList	add(x)	$\Omega(1)$	$\Theta(1)$	$O(1)$	$O(1)$	Direct $O(1)$ node append at head or tail using saved references.
	add(index, x)	$\Omega(1)$	$\Theta(N)$	$O(N)$	$O(1)$	Traversing to index takes $O(N)$ steps; pointer rewiring itself takes $O(1)$ moves.
	remove(index)	$\Omega(1)$	$\Theta(N)$	$O(N)$	$O(1)$	Traversal requires $O(N)$ pointer steps before executing $O(1)$ node unlinking.
	get(index)	$\Omega(1)$	$\Theta(N)$	$O(N)$	$O(1)$	Sequential node traversal (pointer chasing) from head to the targeted index.
	contains(x)	$\Omega(1)$	$\Theta(N)$	$O(N)$	$O(1)$	Linear iteration matching node values sequentially from head to tail.
MinHeap	insert(x)	$\Omega(1)$	$\Theta(\log N)$	$O(\log N)$	$O(1)$	Element added to array end; bubble-up traverses tree height $\log_2 N$ in worst case.
	peekMin()	$\Omega(1)$	$\Theta(1)$	$O(1)$	$O(1)$	Minimum

						element is always located at root node array[0].
	extractMin()	$\Omega(\log N)$	$\Theta(\log N)$	$O(\log N)$	$O(1)$	Root replaced by last leaf; bubble-down traverses tree height $\log 2N$.

2. Formal Loop Invariant Proofs

According to the specification, we formally prove the correctness of two iterative methods.

Proof 1: Linear Search (contains) in DynamicArray

Code Implementation:

```
public boolean contains(int element) {
    for (int i = 0; i < size; i++) {
        counter.addStep();
        counter.addComparison();
        if (data[i] == element) {
            return true;
        }
    }
    return false;
}
```

- Invariant: At the start of iteration i (where $0 \leq i \leq \text{size}$), the target value key is not present in the subarray $\text{data}[0 \dots i - 1]$.
- Initialization: Before the first iteration ($i = 0$), the subarray $\text{data}[0 \dots -1]$ is empty. Vacuously, an empty set contains no elements, so key is not present. The invariant holds.
- Maintenance: Assume the invariant holds at the start of iteration i . During iteration i , the loop evaluates $\text{data}[i]$:
 - If $\text{data}[i] == \text{key}$, the method returns true immediately, terminating with a correct positive match.
 - If $\text{data}[i] \neq \text{key}$, variable i increments to $i + 1$. Since key was absent in $\text{data}[0 \dots i - 1]$ and $\text{key} \neq \text{data}[i]$, key is absent in $\text{data}[0 \dots i]$. The invariant holds for $i + 1$.
- Termination: The loop terminates when $i = \text{size}$ without an early return. By the invariant, key is absent from $\text{data}[0 \dots \text{size} - 1]$ (the entire dynamic array).

-Conclusion: Returning false upon loop completion guarantees that key is not present in the array, proving the operation is correct

Proof 2: Linear Search (contains) in MyLinkedList

Implementation Code:

```
public boolean contains(int element) {  
    Node curr = head;  
    while (curr != null) {  
        counter.addStep();  
        counter.addComparison();  
        if (curr.val == element) {  
            return true;  
        }  
        curr = curr.next;  
    }  
    return false;  
}
```

Invariant: Let $k \geq 0$ be the number of steps traversed from head. At the start of the k -th iteration, current references the k -th node in the list, and key is not present in any of the first k nodes (from index 0 to $k - 1$).

Initialization: Before the first iteration ($k = 0$), current = head (pointing to node 0). Zero nodes have been checked, so the set of inspected nodes is empty. The invariant holds.

Maintenance: Assume the invariant holds before iteration k . The method evaluates current.value:

If current.value == key, it returns true immediately (correct match found).

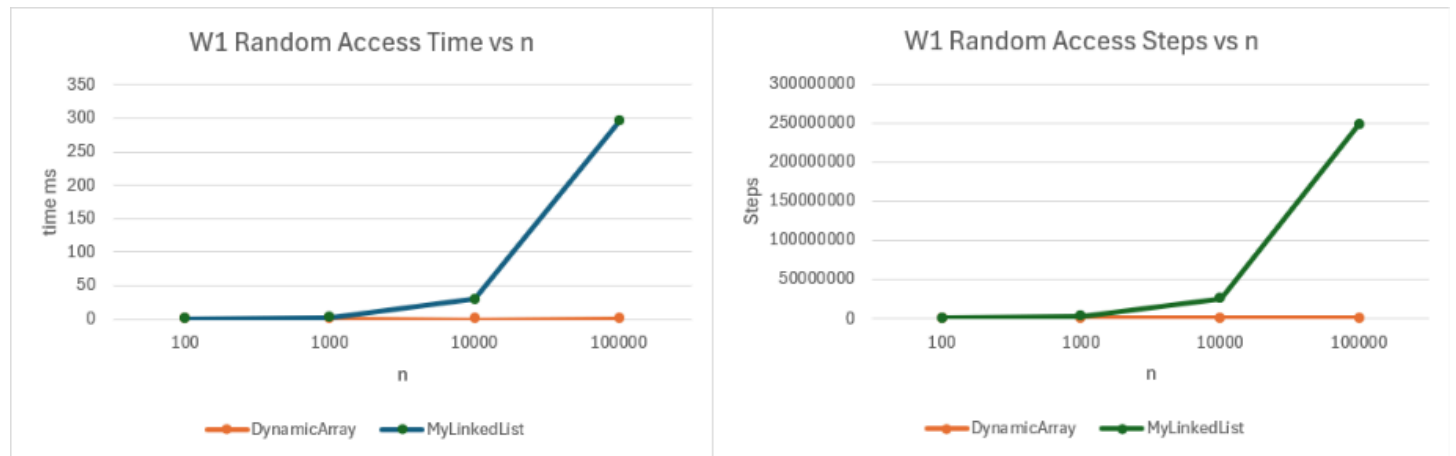
If current.value != key, current advances to current.next ($k \rightarrow k + 1$). Since key was not in the first k nodes and current.value != key, key is not in the first $k + 1$ nodes. The invariant holds for $k + 1$.

Termination: The loop terminates when current == null, corresponding to $k = N$ (all nodes traversed). By the invariant, key is not present in any node from index 0 to $N - 1$.

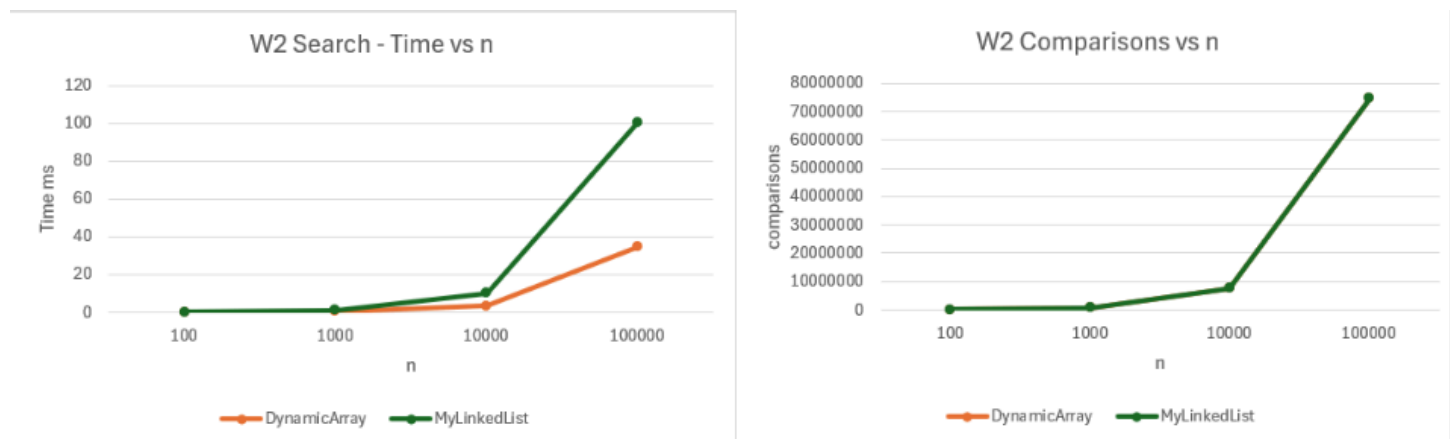
Conclusion: Returning false when current reaches null proves that key does not exist in the linked list, guaranteeing correctness

3. Experimental Workload Results

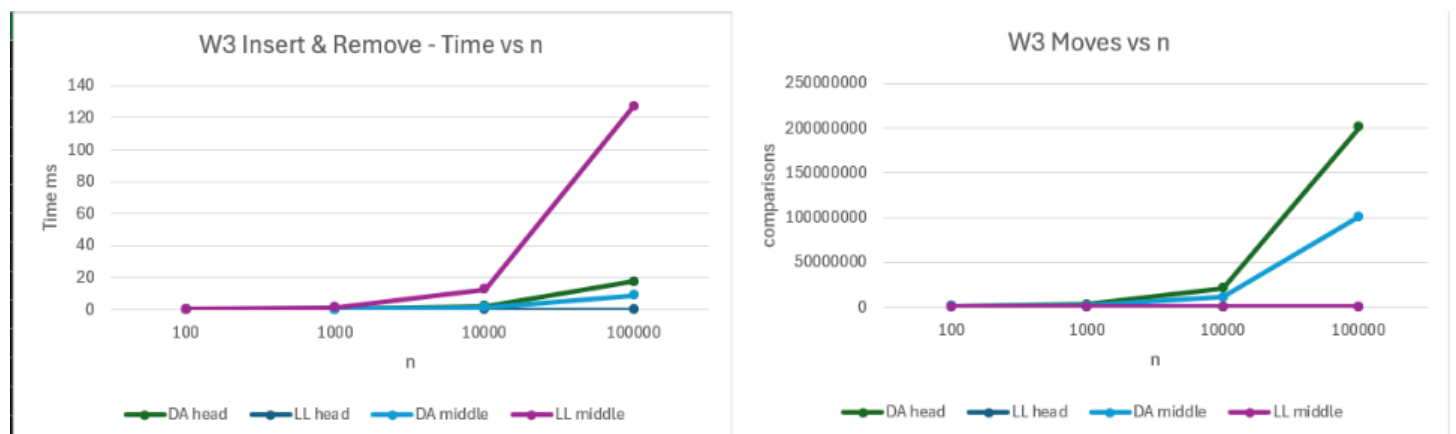
Workload benchmarks were measured across sizes N in $\{100, 1000, 10000, 100000\}$ using median values over 5 runs with a warm-up phase



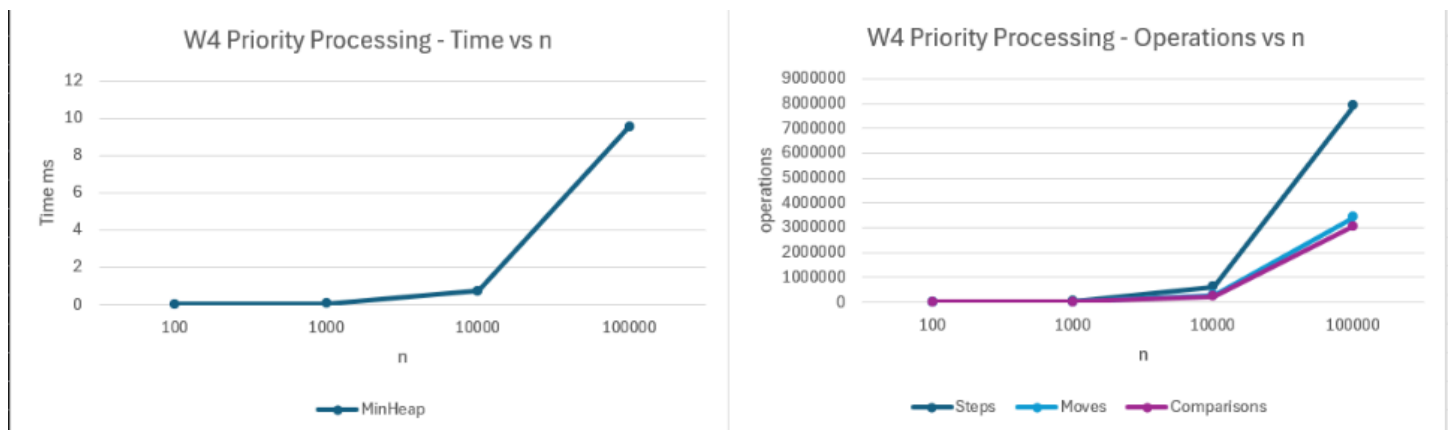
W1_RandomAccess



W2_Search



W3_InsertRemove



W4_PriorityProcessing

4. Discussion

Although `DynamicArray` and `MyLinkedList` share the same $O(N)$ asymptotic complexity for linear searches (`contains`), `DynamicArray` performs significantly faster in real execution time. This performance gap is explained by hardware memory architectures:

1) **Contiguous Allocation & CPU Cache Lines:** `DynamicArray` stores primitive `int` values in a contiguous memory block (`int[]`). When the CPU requests a single memory address, the memory controller prefetches an entire 64-byte cache line into L1/L2 caches. Subsequent accesses read directly from low-latency cache rather than main memory.

2) **Pointer Chasing & Cache Misses:** `MyLinkedList` allocates distinct `Node` instances across non-contiguous heap memory. Traversing the list requires "pointer chasing" (`current = current.next`), forcing frequent CPU cache misses and stalling execution while fetching data from RAM.

3) **Object Overhead:** In 64-bit JVMs, every node object carries a 12–16 byte header plus reference pointers. Storing a single 4-byte `int` requires 24–32 bytes of memory, inflating memory usage and placing additional load on the Garbage Collector.

Practical Selection Criteria:

-`DynamicArray`: Ideal for random access (`get(i)`), linear iterations, and read-heavy workloads where cache efficiency dominates.

-`MyLinkedList`: Beneficial when constant time $O(1)$ insertions or deletions occur exclusively at boundaries (head/tail) without index traversal.

-`MinHeap`: Optimal for priority scheduling where fast access to extreme values ($O(1)$ `peekMin`) and logarithmic updates ($O(\log N)$) are required